0x28 immediately after the replacement character bytes: this is because *0x28* is a valid UTF-8 byte. `mbstring` has treated *0xC3* as the sole invalid byte, because *0x28* is a valid character.

Here's another example:

```
mb_substitute_character(0xFFFD);
$invalidSurrogatePair = "\xD8\x00\xD8\x00";
$validUtf16 = "\x00\x54\x00\x65\x00\x73\x00\x74";

$converted = mb_convert_encoding(
    $validUtf16 . $invalidSurrogatePair,
    to_encoding: 'UTF-8',
    from_encoding: 'UTF-16BE',
);
print $converted . PHP_EOL;
print bin2hex($converted);
```

```
Output:
Test��
54657374efbfbdefbfbd
```

This time we're creating some broken UTF-16BE by appending a broken surrogate pair onto the string "Test". Surrogate pairs in UTF-16 are the mechanism used to encode Unicode codepoints above U+FFFF, which require more than two bytes, just like we've seen several examples of multi-codepoint characters (such as bald men and flags). Without diving too deeply into UTF-16, just know that the sequence U+D800 U+D800 is invalid because U+D800 can only be followed by codepoints U+DC00 – U+DFFF. This time we get two substitute U+FFFD characters in the output, UTF-8 encoded as *0xEF 0xBF 0xBD*, because the invalid pair is treated as two distinct characters. You'll get wrinkles like this when characters are substituted due to invalid encoding; the main purpose of the mechanism isn't to make output strings look pretty but to make them safe. What's broken in one encoding might be perfectly valid in another, and indeed there have been exploits in the past that took advantage of this to sneak problem meta-characters (such as directory separators) past validation layers.

`mb_list_encodings()`, `mb_encoding_aliases()`